



۸۱۰۱۰۱۴۹۲

شبکه‌های کامپیوتری
نام و نام خانوادگی: علیرضا کریمی



نوت ۳۰ آذر

فصل ششم: *link layer* اسلاید ۱ تا ۲۴ [۱]

لینک‌ها می‌توانند به صورت *wired* یا *wireless* یا *LAN* باشند. روی لینک‌های مختلف می‌توانیم پروتکل‌های مختلف لینک داشته باشیم. که هر پروتکل سرویس‌های متفاوتی ارائه می‌دهد.

framing

اینکه *datagram* لایه شبکه را در *frame* که *pdu* لایه دوم است، *encapsulate* کنیم و هدر و *trailer* (که نمیدونم چیه) به آن اضافه کنیم. در هدر مثلاً آدرس *MAC* مبدا و مقصد قرار دارد که این آدرس متفاوت از آدرس *IP* است. سرویس‌های مختلفی داریم. مثلاً سرویس *reliable delivery* که مثل همان *rdt* هست، اینجا هم این سرویس هست. برای لینک‌های فیبر چون خطای بیتی کم است نیاز به این سرویس نیست. ولی برای *wireless* که *high error rates* دارد، نیاز به این سرویس هست.

بحث *flow control* که یک هماهنگی میان مبدا و مقصد شکل می‌دهد، اینجا نیز هست. سرویس‌های *error detection & correction* هم هست. اینکه لینک‌های ما *half duplex* باشند که یعنی در یک لحظه فقط یک طرف می‌تواند بفرستد و یا اینکه *full duplex* باشد که یعنی همزمان هم می‌تواند بفرستد هم دریافت کند.

بخشی از لایه *link* در *cpu* پیاده ساز میشود به همراه لایه‌های بالاتر و بخشی از لایه لینک هم به همراه لایه فیزیکی در *adaptor* یا همان *Network interface card = NIC* کارت شبکه یا *chip* پیاده سازی میشود. درواقع این لایه به نوعی ترکیب سخت افزار و نرم افزار و *firmware* است. علاوه بر اینکه اون *datagram* رو در فریم *encapsulate* کردیم، یک سری بیت هم مربوط به *error checking* و *flow control* و *rdt* و غیره (مثل آدرس‌های مک) اضافه میکنیم بهش.

بحث $EDC = error\ detection\&\ correction$

اینجا می‌گه یک سری بیت منسوب به EDC به کل داده D مون اضافه میکنیم. (فرض کنید $\langle D, EDC \rangle$ داریم) در مقصد ممکنه به خاطره خطای لینک، $\langle D', EDC' \rangle$ به مقصد میرسد، حالا ممکن است بتوانیم ارور را تشخیص دهیم به کمک EDC که آیا همه بیت های D' درست هستند مثل D که در این صورت $error\ detected$ میشه و در غیر اینصورت میشه $no\ error\ detected$ و ارور تشخیص ندادیم، نه اینکه ارور نداریم.

انواع EDC

کلا هیچ روش صد در صدی برای تشخیص خطا نداریم. ولی کلا هر چی تعداد بیت های EDC بیشتر باشد، احتمال تشخیص و درست کردن خطا میتواند بیشتر شود.

روش $parity\ checking$

خود این رو نوع دارد مثلاً:

- $single\ bit\ parity$: توی این حالت EDC ما کلا یک بیت هست که همون $parity$ تمام بیت های توی D هست. این نوع EDC اگر یک یا تعداد فرد بیت فلیپ شده باشد را تشخیص میده اما نمیتواند آن را اصلاح کند.
- $two - dimetional\ bit\ parity$ در این حالت اون D رو به صورت سطر و ستون در میاریم مثلاً i سطر در j ستون و برای هر سطر و برای هر ستون $parity$ را حساب میکنیم. پس در اینجا به $i + j$ بیت برای $parity$ نیاز داریم فکر کنیم (البته مطمئن نیستیم) اینجا علاوه بر تشخیص خطا، امکان اصلاح خطا هم تا حدی داریم. برای حالتی که یک بیت یا دوبیت یا حتی ۳ بیت تغییر میکند، امکان تشخیص و اصلاح داریم ولی برای حالتی که ۴ تا بیت روی رئیس یک مستطیل دارای خطا میشوند، امکان تشخیص و اصلاح نداریم.

روش $checksum$

این روش رو توی لایه انتقال هم قبلاً داشتیم و اینجا هم داریم. به صورت ۱۶ بیت ۱۶ بیت، اعداد رو با هم جمع مکمل ۱ میکنیم و خروجی رو در ۱۶ بیت بخش $checksum$ قرار میدهیم. باز در اینجا اگر گیرنده متوجه خطا با $checksum$ شد که خطا رو تشخیص دادیم ولیی اگر نشد، $no\ error\ detected$ هست و نه اینکه خطا نداریم.

روش $CRC = Cyclic\ redundancy\ check$

این روش یک روش قوی در $error\ detection$ هست که به صورت گسترده در $ETHERNET$ و 802.11 و ATM و $WiFi$ استفاده میشود.

انگار برای تشخیص خطاهای کمتر از $r + 1$ کاربرد دارد. ممکنه خطاهای بیشتر از اون تعداد رو پوشش نده. حالا چیزی که فهمیدیم از این روش اینه که ما یک $G = \text{generator}$ داریم که $r + 1$ بیتی هست. داده اصلی مون هم همون D هست با d بیت که میخوایم از سمت راستش یک داده r بیتی R رو بهش بچسبونیم concat کنیم که در نهایت اون $\langle D, R \rangle$ بر G بخش پذیر باشد.

در واقع میدونیم که $\langle D, R \rangle = D \times 2^r \text{ XOR } R = nG$ حالا اگه دو طرف رو با R دوباره XOR کنیم، چون اون $\langle D, R \rangle$ اون بیت های lsb ش دارن دوباره با خود XOR R میشن پس اون تیکه اش میشه صفر. پس داریم: $D \times 2^r = R \text{ XOR } nG$ پس یعنی اگر ما $D \times 2^r$ را بر G تقسیم کنیم، باقی مانده آن همان R خواهد بود. والسلام حالا که R را بدست آوردیم، اون $\langle D, R \rangle$ رو در مقصد میسازیم و میفرستیم و در گیرنده که خودش G رو داره، میاد و اون $\langle D, R \rangle$ رو بر G تقسیم میکنه، اگر باقی مانده اش صفر شد، باز یعنی no error detected و اگر غیر صفر شد یعنی error detected .

روش تقسیم رو هم در اسلاید ۱۶ گفته است.

پروتکل های multiple access

ببینید ما دو نوع لینک داریم:

۱. لینک های point-to-point مثل PPP برای دستیابی های dial-up و تلفنی و یا مثلا point-to-point های موجود در ETHERNET بین سویچ ها و هاست ها.

۲. لینک های broadcast یا shared مثلا در ETHERNET قدیمی و old-fashion یا در upstream HFC یا در wireless حالا

وقتی که چند تا نود میخوان روی یک bus مشترک با هم صحبت کنند، ممکن است همزمان دو نفر سیگنال بفرستند و با بقیه روی آن باس صحبت کنند که این باعث تداخل سیگنال ها و collision میشود. برای اجتناب از آن، پروتکل هایی آمده است که یا احتمال آن collision را صفر یا کم میکند یا اینکه امکان recovery از آن collision را به ما میدهد.

این نوع پروتکل ها به گونه ای باید عمل کنند که پیام های کنترلی شون هم از همون لینک shared استفاده کنه و نه اینکه $\text{out-of-bound management}$ و یک لینک جدا برای کنترل داشته باشه.

این پروتکل در حالت ایده آلش مثلا چند تا ویژگی میتونه داشته باشه:

- اگه فقط یک نود میخوان transmit کنه، از کل ریت R استفاده کنه.
- اگه M تا نود میخوان داده جابجا کنن، به طور میانگین در ریت R/M جابجا کنن

- الگوریتم به صورت *fully decetralized* باشد یعنی اینجوری نباشه که مثلا *parameter server* یا یک *coordinator* داشته باشیم برای انتقال ها و یا اسلات ها و *timing* رو بخوان با هم سینک کنند نود ها.
- الگوریتم ساده باشد

ولی خب همیشه همه این ها رو همزمان با هم داشت. هر کدوم از پروتکل ها بخشی از این موارد رو رعایت نکردند. بعضی از این پروتکل های *multiple access* عبارتند از:

۱. *channel partitioning* که میاد لینک مشترک رو از لحاظ پهنای باند یا *timing* یا کد، تقسیم میکنه بین نود ها
۲. روش *random access* که لینک رو تقسیم نمیکنه و اجازه *collision* رو میده ولی امکان *recovery* از اون *collision* رو داره در خودش.
۳. روش هاس *taking turns* که هر کی نوبت میگیره و در نوبت خودش، از لینک استفاده میکنه. و خب اون نود های که داده بیشتری برای ارسال دارند، نوبت طولانی تری دارن یا میگیرن.

روش های *channel partiitoning*

TDMA = time division multiple access

با اون *TDM = time division multiplexing* قاطی نکنیا. اون یه لایه دیگه بود. اینجا میاد زمان رو به یک سری *frame* تقسیم میکنه و هر فریم رو بر اساس تعداد نفرات به اون تعداد *slot* تقسیم میکنه. مدت زمان هر اسلات رو هم میزازه مساوی مدت زمان ارسال یک بسته. بدی این روش اینه که میتونیم اسلات های *idle* داشته باشیم اگه اون نود هیچی نفرسته اون موقع

FDMA = frequency division multiple access

اینجا پهنای باند رو بین اون تعداد نفر به صورت *fair* احتمالا ولی خب به بخش های مشخص تقسیم میکنه و به هر کسی فقط یک بخشی میده. اینجوری هم اگه یک نود چیزی نفرسته، از پهنای باندش استفاده نکرده و کلا *utilization* خوبی نداره چون یک سری باند های فرکانسی *idle* هستن.

روش های *random access*

توی این روش ها خوبیش اینه که میتونیم از کل پهنای باند استفاده کنیم و اون مشکل *idle* گذاشتن لینک رو اینجا نداریم. اینجا در ریت کامل لینک همون R میایم *transmit* میکنیم ولی چون هیچ *coordinator* نیست، ممکنه *collision* رخ بده که باید اون رو *recovery* کنیم.

پس نکته اینه که چجوری *collision* رو تشخیص بدیم و چجوری اون رو *recover* کنیم. مثلا با بازارسال سیگنال با تاخیر

از انواع پروتکل‌های *random access MAC protocol* عبارتند از *pure ALOHA* و *slotted ALOHA* و *CSMA*, *CSMA/CD*, *CSMA/CA* و غیره. اینجا *MAC* یعنی *multiple access control* ولی اون آدرس مک میشد *media access control*

مراجع

[1] “ch6-slide-1-to-24,” <https://drive.google.com/drive/folders/>, 2022.